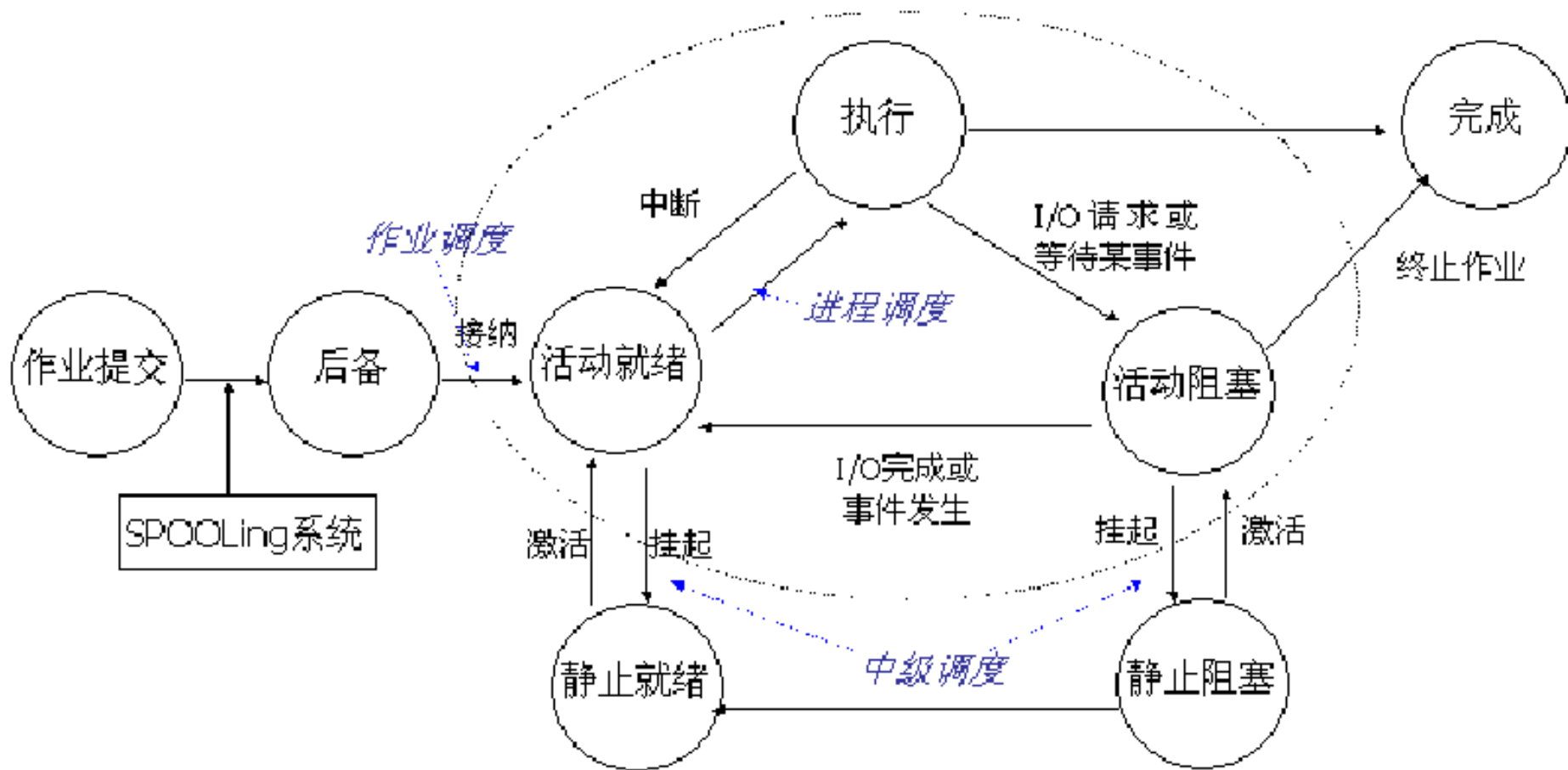


第二章总结

- 前趋图：画图、信号量编程
- 进程的结构特征和三个特性、定义，P37-38
- 进程状态：带挂起的状态图P39、状态转换使用的原语
- 进程控制块PCB包含的四方面信息
- 进程控制：创建过程、终止过程
- 进程同步：主要任务P47，同步机制原则P50
- 四种信号量特点
- 前趋关系的信号量编程
- 生产者-消费者问题：记录型信号量的编程
- 读者-写者问题：信号量集的编程
- 高级进程通信：定义P65，三种通信类型，特点
- 线程属性：轻型实体、独立调度和分派的基本单位、可并发执行、共享进程资源

3.1 处理机调度的基本概念



三级调度之间联系

3.1 处理机调度的基本概念

1. 高级调度

- 功能：根据调度算法和计算机状态，从外存选择一个或多个作业调入内存
 - 创建进程、分配内存等资源、将进程放入就绪队列等待CPU调度
- 作业和作业步
- 作业控制块JCB
- 作业调度
 - 接纳多少个作业：
 - 周转时间：作业太多增加平均周转时间，影响服务质量
 - 吞吐量：作业太少降低资源利用率
 - 接纳哪些作业：调度算法

3.1 处理机调度的基本概念

2. 低级调度

- 低级调度即进程调度，它决定哪个进程获得**CPU**
- 功能：保存现场，调度新进程、新进程获取**CPU**控制权
- 三个基本机制：排队器、分派器、上下文切换机制
- 非抢占方式：正在执行的进程运行完毕或发生阻塞，才把**CPU**分配其他进程，主动让出**CPU**
 - 优点：实现简单、系统开销小
 - 缺点：难以满足紧急任务的要求，例如实时任务
 - 适用于大多数的批处理系统环境，不适合实时系统
- 抢占方式：允许暂停当前执行的进程，重新分配**CPU**给另一进程
 - 优先权原则、短作业(进程)优先原则、时间片原则

3.1 处理机调度的基本概念

3. 中级调度

- 主要目的是为了提高内存利用率和系统吞吐量
- 操作：为了使得暂时不能运行的进程不占用宝贵的内存资源，将它们调至外存
- 挂起和唤醒
- 主要使用存储器管理的对换功能

3.1 处理机调度的基本概念

3.1.1 高级、中级和低级调度

■ 高级调度：以作业为调度对象

- 从外存到内存
- 进程创建操作

■ 中级调度：以进程为调度对象

- 外存与内存互换
- 挂起与唤醒操作

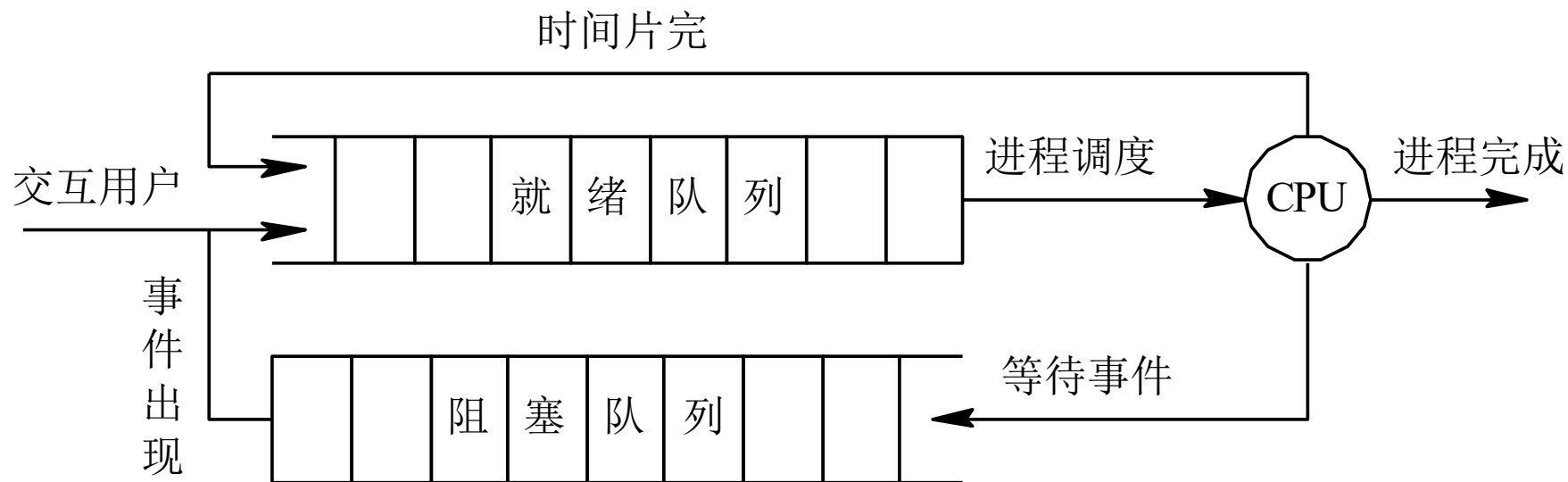
■ 低级调度：以进程为调度对象

- 一直在内存中
- 分配**CPU**执行程序

3.2 调度队列模型和调度准则

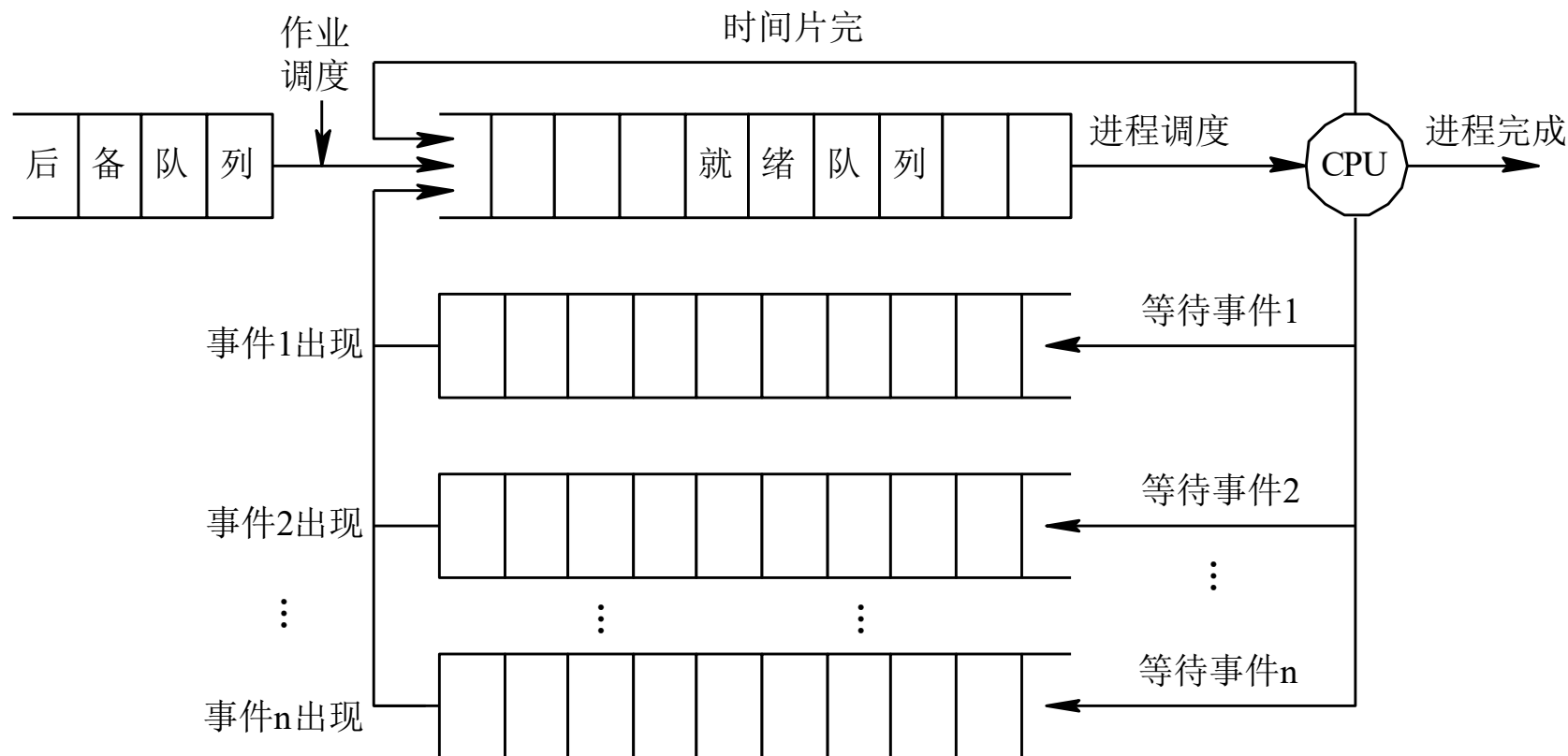
3.1.2 调度队列模型

1. 仅有进程调度的调度队列模型：CPU调度先到先得



3.2 调度队列模型和调度准则

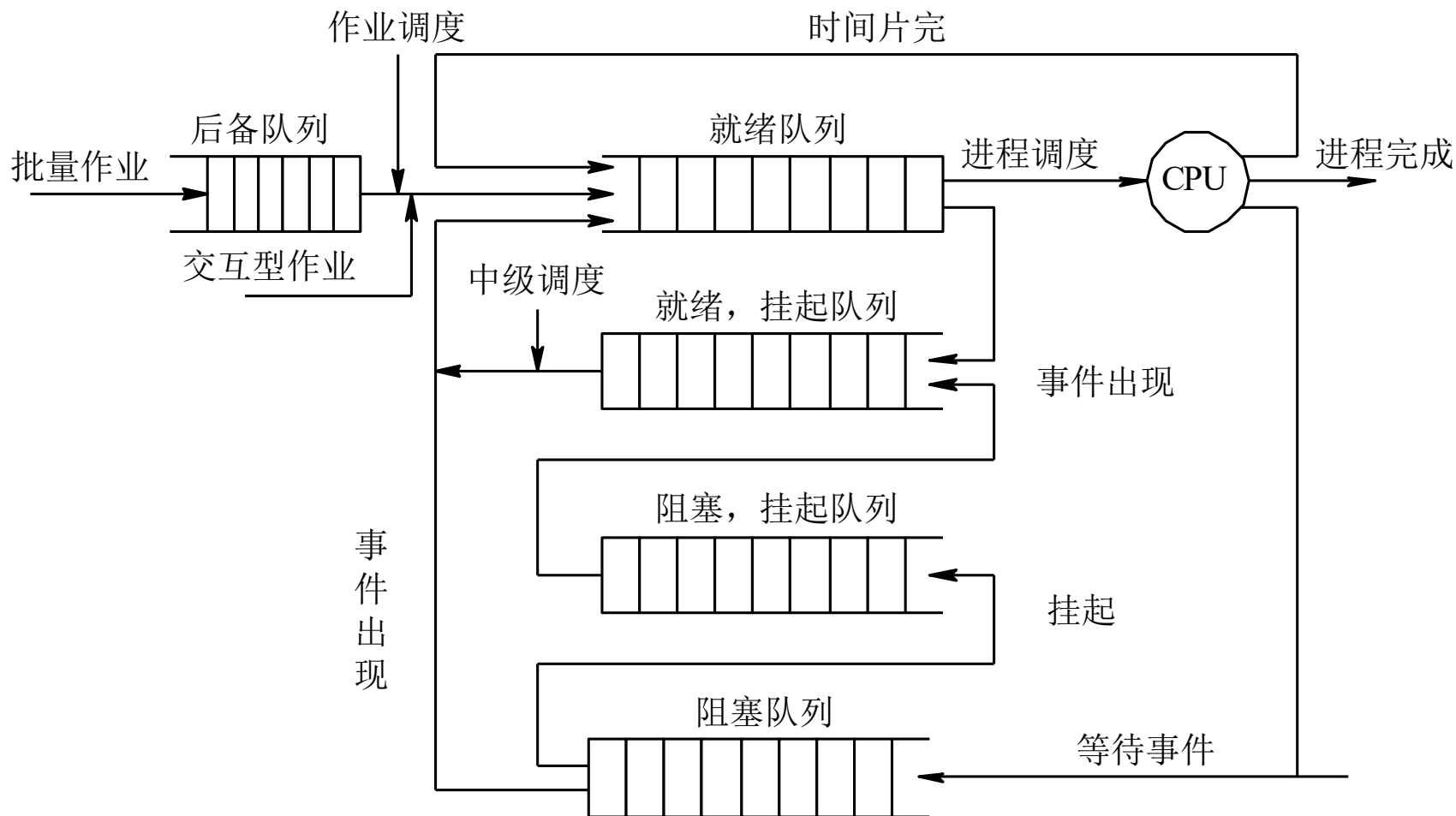
2. 具有高级和低级调度的调度队列模型



■ CPU调度采用优先权，设置多个阻塞队列

3.2 调度队列模型和调度准则

3. 同时具有三级调度的调度队列模型



3.2 调度队列模型和调度准则

选择调度方式和调度算法的若干准则

■ 面向用户的准则

□ 周转时间短：平均周转时间、平均带权周转时间

■ 周转时间：作业进入到完成的时间

■ 带权周转时间：周转时间与服务时间的比例

$$T = \frac{1}{n} \left[\sum_{i=1}^i T_i \right]$$

□ 响应时间快

□ 截止时间的保证

□ 优先权准则

$$W = \frac{1}{n} \left[\sum_{i=1}^n \frac{T_i}{T_{Si}} \right]$$

■ 面向系统的准则

□ 系统吞吐量高

□ 处理机利用率好

□ 各类资源的平衡利用

3.3 调度算法

- 处理机调度实质是一种资源分配
- 调度算法含义：P91
- 关键是资源分配策略的制定
- 目标不同、系统环境不同，应用不同的算法
- 固定属性
 - 到达时间：进入系统的时间
 - 开始时间：首次使用**CPU**的时间
 - 服务时间：需要使用**CPU**的时间，又叫运行时间
 - 完成时间：退出系统的时间
 - 周转时间：完成时间减去到达时间，包含服务时间和等待时间，若等待时间为**0**即没有等待
 - 平均周转时间：周转时间/服务时间

3.3 调度算法

先来先服务调度算法

- 作业调度：按时间先后顺序，选择最先来的一个或多个作业
- 进程调度：从就绪队列中，选择最先进入队列的进程，即队首进程。
 - 采用非抢占式调度
- 优点：实现简单，算法开销小，有利于长时间作业（进程）
- 缺点：不利于短时间作业（进程）

3.3 调度算法

先来先服务调度算法

进程名	到达时间	服务时间	开始执行时间	完成时间	周转时间	带权周转时间
A	0	1	0	1	1	1
B	1	100	1	101	100	1
C	2	1	101	102	100	100
D	3	100	102	202	199	1.99

3.3 调度算法

短作业(进程)优先调度算法

- 优先选择运行时间最短的作业或进程
- 非抢占式调度
- 优点：有利于短作业或短进程
- 缺点：导致长作业或进程长时间不被调度，不能保证实时性，执行时间一般基于用户估算，准确性不足

3.3 调度算法

短作业(进程)优先调度算法

调度算法 \ 作业情况	进程名	A	B	C	D	E	平 均
	到达时间	0	1	2	3	4	
	服务时间	4	3	5	2	4	
FCFS (a)	完成时间	4	7	12	14	18	
	周转时间	4	6	10	11	14	9
	带权周转时间	1	2	2	5.5	3.5	2.8
SJF (b)	完成时间	4	9	18	6	13	
	周转时间	4	8	16	3	9	8
	带权周转时间	1	2.67	3.1	1.5	2.25	2.1

3.3 调度算法

高优先权优先调度算法

- 优先权调度算法的类型

- 非抢占式优先权算法

- 抢占式优先权调度算法

- 优先权类型

- 静态优先权：创建进程时已确定并在运行过程中不变

- 相关因素：进程类型、进程对资源的需求、用户要求

- 动态优先权：随进程的推进或等待时间增加而改变

- 例如等待时间越长则提高优先权

3.3 调度算法

■ 高响应比优先调度算法

$$\text{优先权} = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = \frac{\text{响应时间}}{\text{要求服务时间}}$$

- 在单处理机环境下，对4个作业A、B、C、D进行非抢占式调度，它们的到达时间分别为：**800、850、900、950**，运行时间为：**200、50、10、20**。假设其他系统开销忽略不计
 - 试用高响应比优先调度算法，写出调度顺序、完成时间、周转时间和带权周转时间
 - 如果在第**1005**秒有新作业E加入，运行时间为**15**。用高响应比优先调度算法，写出调度顺序、完成时间、周转时间和带权周转时间

3.3 调度算法

- 高响应比优先调度算法的优点：
 - ① 对于等待时间相同的时候，服务时间愈短则优先权愈高，算法有利于短作业
 - ② 对于服务时间相同的时候，等待时间愈长其优先权愈高，算法实现的是先来先服务。
 - ③ 对于长作业，作业的优先级可以随等待时间的增加而提高，当其等待时间足够长时，其优先级便可升到很高保证能获得处理机。
- 缺点：每次调度前都要计算优先权，增加系统开销

3.2 调度算法

- 基于时间片的轮转调度算法
 - 将所有就绪进程按先来先服务排成队列
 - 把**CPU**分配给队首进程，进程只执行一个时间片
 - 时间片用完，**OS**通过计时器发出时钟中断，停止进程
 - 将已使用时间片的进程送往就绪队列的末尾
 - 分配处理机给就绪队列中下一进程
- 时间片大小的设定影响系统性能
 - 实例 **P95**

3.2 调度算法

- 在单处理机环境下，对4个作业A、B、C、D进行非抢占式调度，它们的到达时间分别为5、7、9、11，所需运行时间分别为10、16、4、8。假设其他系统开销忽略不计
 - 采用时间片轮转算法，时间片长度为3，写出调度过程。
 - 如果在第18秒有新作业E进入系统，运行时间为5。写出调度过程。

时间片	执行进程	时间片	执行进程	时间片	执行进程
1	A	7	C完成	13	E完成
2	B	8	D	14	A完成
3	C	9	E	15	B
4	D	10	A	16	B
5	A	11	B	17	B完成
6	B	12	D完成		